



ICT306 Advanced Cybersecurity

WEEK 2(A) Hacking Webservers

A deep dive into web server vulnerabilities, attack methodologies, and the tools used by ethical hackers to identify and mitigate risks in real-world environments.

Prescribed Resources

Module 13

This week's content maps directly to **Module 13** of the CEH v12 study guide. Work through all reading and hands-on activities before the next session.

Required Textbook

Meredith, D. (2022). *Certified Ethical Hacker (CEH) v12 312-50 Exam Guide: Keep up to date with ethical hacking trends and hone your skills with hands-on activities.* Packt Publishing Ltd.

This text provides both theoretical foundations and practical, lab-based exercises aligned with the CEH certification exam objectives.

Learning Outcomes

By the end of this lecture, students should be able to:

1

Explain

The architecture and operation of common web servers such as Apache and IIS.

2

Identify

Key vulnerabilities and misconfigurations that expose web servers to attacks.

3

Describe

Major web server attack types including DoS/DDoS, DNS hijacking, directory traversal, and MitM attacks.

4

Demonstrate

The use of ethical hacking tools for reconnaissance, vulnerability scanning, and penetration testing.

5

Evaluate

The effectiveness of defense mechanisms such as patch management, access control, and network segmentation.

6

Apply

Appropriate countermeasures and security tools to detect, prevent, and mitigate web server attacks.

Module Objectives

This module is structured in two parts — foundational attack knowledge followed by defensive strategies and penetration testing practice.

Part 1 — Attack Concepts

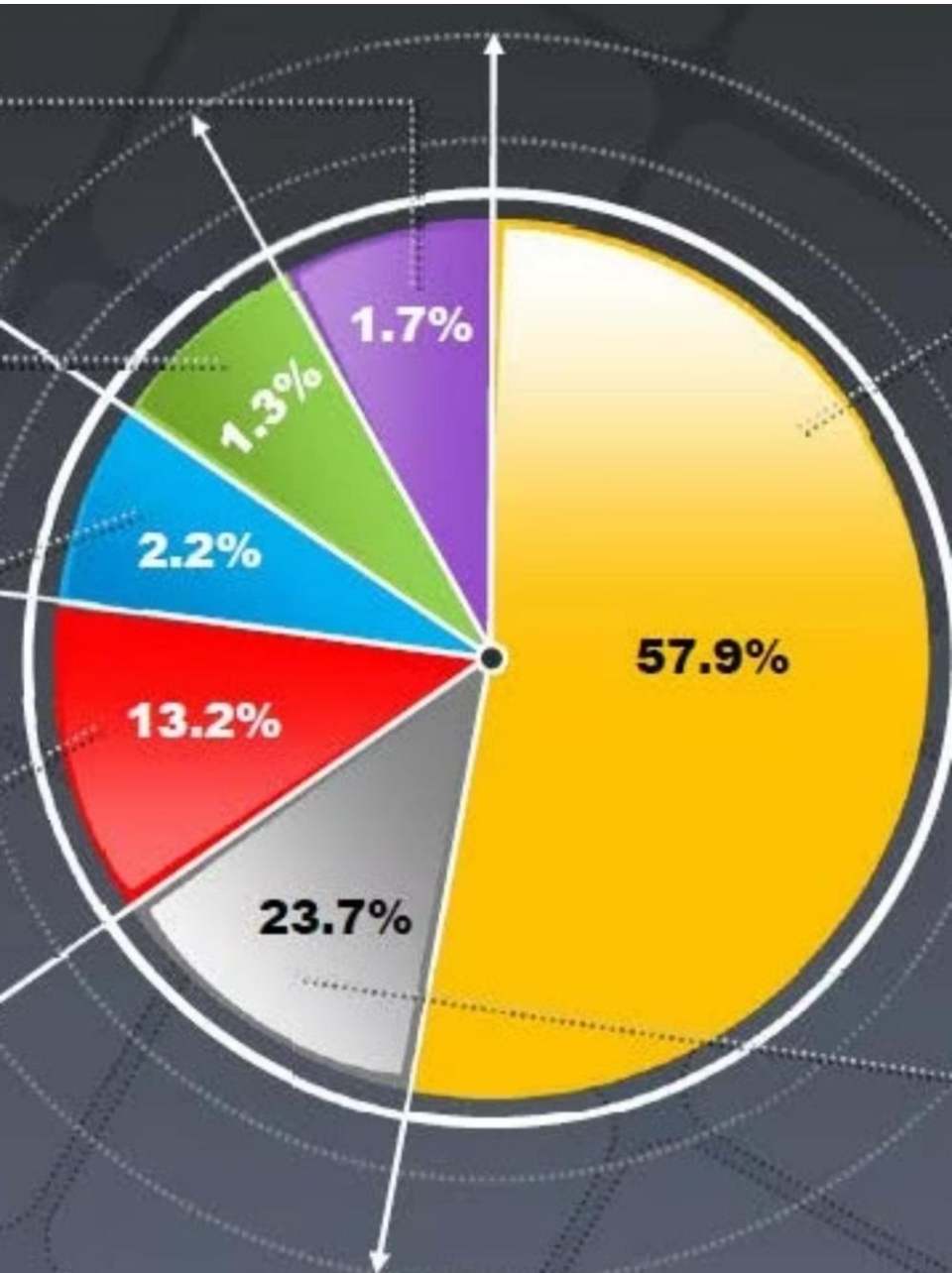
- Understanding Webserver Concepts
- Understanding Webserver Attacks
- Understanding Webserver Attack Methodology
- Webserver Attack Tools

Part 2 — Defense & Testing(Self Study)

- Countermeasures against Webserver Attacks
- Overview of Patch Management
- Webserver Security Tools
- Overview of Webserver Penetration Testing

Web Server Market Shares

Understanding which web servers dominate the internet helps ethical hackers and security professionals prioritize their knowledge. Apache leads the market by a wide margin, making Apache vulnerabilities among the most impactful to study.



57.9%

Apache

Dominant open-source web server used across Linux/Unix environments worldwide.

23.7%

Nginx

High-performance server favored for reverse proxying and load balancing.

13.2%

Microsoft IIS

Enterprise-grade server integrated with Windows Server environments.

2.2%

LiteSpeed

Commercial server known for speed and Apache drop-in compatibility.

Web Server Security Issues

A web server is a program — both hardware and software — that hosts websites. Attackers typically target **software vulnerabilities** and configuration errors to compromise them. While network and OS-level attacks can be defended with firewalls and IDS, web servers remain accessible from anywhere on the internet, making them inherently **more vulnerable**.

- ❏ The layered stack model (Security → Network → OS → Web Server → Database → Third Party → Custom App) illustrates how attackers can target any tier. Weaknesses at higher layers often bypass controls at lower ones.



Why Web Servers Are Compromised

Configuration & Access Issues

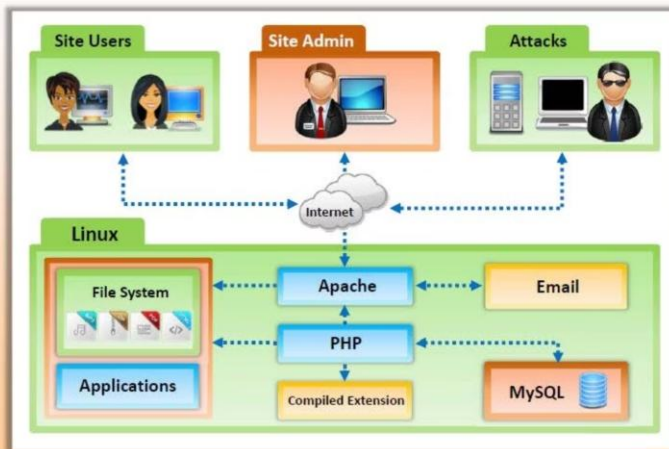
- Improper file and directory **permissions**
- Server installation with **default settings**
- Enabling **unnecessary services** (CMS, remote admin)
- Security conflicts with business ease-of-use
- Lack of proper security policies and maintenance
- Improper authentication with external systems
- **Default accounts** with default or no passwords

Software & Certificate Flaws

- Unnecessary default, backup, or **sample files**
- Misconfigurations in web server, OS, and networks
- Bugs in server software, OS, and web applications
- **Misconfigured SSL certificates** and encryption settings
- Administrative or **debugging functions** left enabled
- Use of **self-signed certificates** and default certificates
- Not using a dedicated server for web services

Open Source Web Server Architecture

The LAMP stack (Linux, Apache, MySQL, PHP) is the foundation of a vast portion of the web. Understanding its architecture reveals how each component interacts — and where attackers can intervene. Site users, administrators, and attackers all share the same internet-facing entry point into the Apache layer.



Linux OS

Provides the file system, process management, and underlying security controls.

Apache

Handles HTTP requests, routing traffic to PHP applications and static files.

PHP

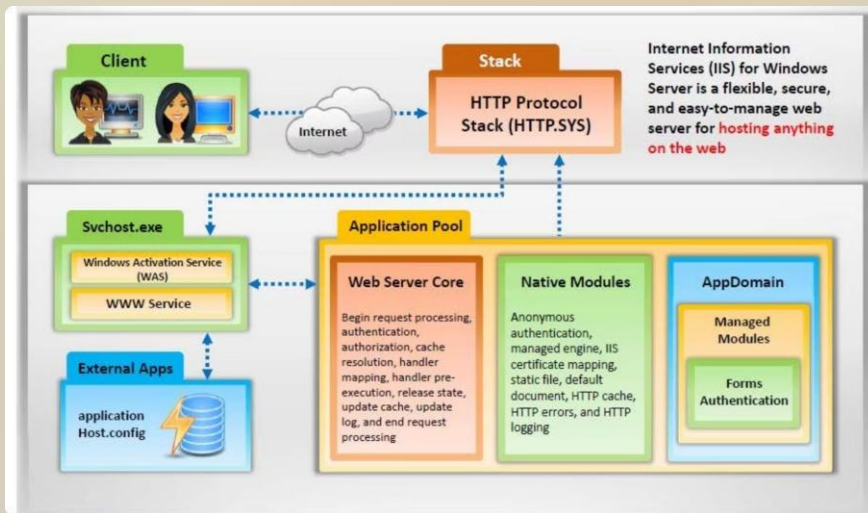
Server-side scripting language that processes dynamic content and database queries.

MySQL

Relational database storing user data, application state, and sensitive records.

IIS Web Server Architecture

Internet Information Services (IIS) for Windows Server is a flexible, secure, and easy-to-manage web server. Its modular architecture separates concerns across the HTTP Protocol Stack, Application Pool, and native/managed modules — providing granular control but also multiple potential attack surfaces.



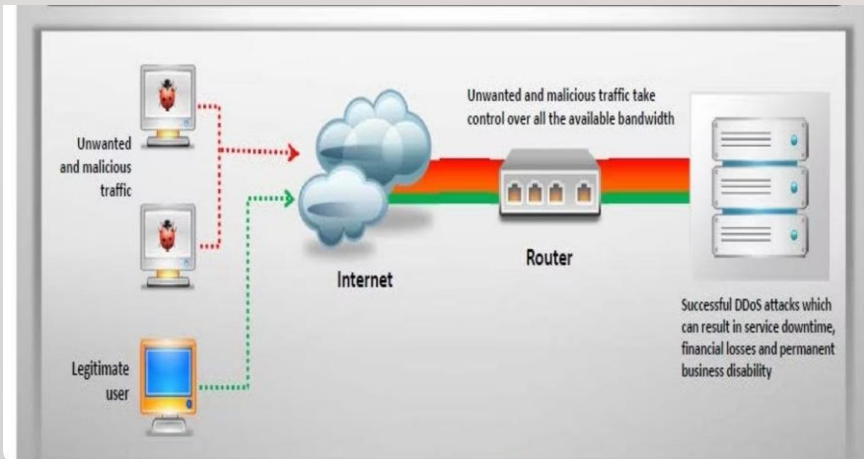
Key Components

- **HTTP.SYS** — Kernel-mode HTTP listener
- **SvcHost.exe / WAS** — Windows Activation Service managing app pools
- **Web Server Core** — Handles authentication, caching, handler mapping

Module Types

- **Native Modules** — Anonymous auth, IIS certificate mapping, HTTP cache
- **Managed Modules** — .NET-based modules in the AppDomain
- **Forms Authentication** — Credential-based access control for web apps

DoS / DDoS Attacks Against Web Servers



How It Works

Attackers flood the web server with numerous **fake requests**, causing the server to crash or become unavailable to legitimate users. Malicious traffic saturates all available bandwidth at the router level.

High-Value Targets

Attackers deliberately target **high-profile web servers** — banks, credit card payment gateways, and government services — to disrupt operations or steal user credentials. Successful DDoS attacks result in service downtime, financial loss, and **permanent business disability**.

Common DoS/DDoS Attack Tools (1–5)

Ethical hackers must understand the tools used in DoS/DDoS attacks to effectively test and defend web infrastructure. The following tools are widely referenced in penetration testing literature and CEH study materials.

1. LOIC — Low Orbit Ion Cannon

One of the most well-known DDoS tools. Floods a target with TCP, UDP, or HTTP requests. Often used in coordinated "hacktivism" campaigns. Available at GitHub.

2. XOIC

A simpler DDoS tool that attacks a target with ICMP, UDP, TCP, or HTTP messages. Designed for ease of use with a basic GUI interface.

3. HULK — HTTP Unbearable Load King

Generates unique obfuscated traffic to bypass caching engines and hit the web server directly with HTTP GET floods. Available on Packet Storm Security.

4. DDOSIM — Layer 7 DDoS Simulator

Simulates several zombie hosts that create full TCP connections to the target server and send application-layer (Layer 7) DDoS requests. Hosted on SourceForge.

5. R-U-Dead-Yet (RUDY)

A slow-POST attack tool that sends HTTP POST requests with an extremely slow body to keep connections open and exhaust server resources.

Common DoS/DDoS Attack Tools (6–10)

6. Tor's Hammer

A slow-POST attack tool that can be run through the Tor network to anonymize the attacker. Sends HTTP POST requests at a very slow rate to exhaust server threads.

7. PyLoris

A Python-based implementation of the Slowloris attack. Targets the connection pool of a server by maintaining many open, slow HTTP connections simultaneously.

8. OWASP DOS HTTP POST

An OWASP-maintained tool that tests server resilience against slow HTTP POST attacks. Useful for security assessment and load-testing within authorized environments.

9. DAVOSET

DDoS Attacks via Other Sites Execution Tool. Leverages third-party vulnerable websites to launch reflected DDoS attacks against a target.

10. GoldenEye

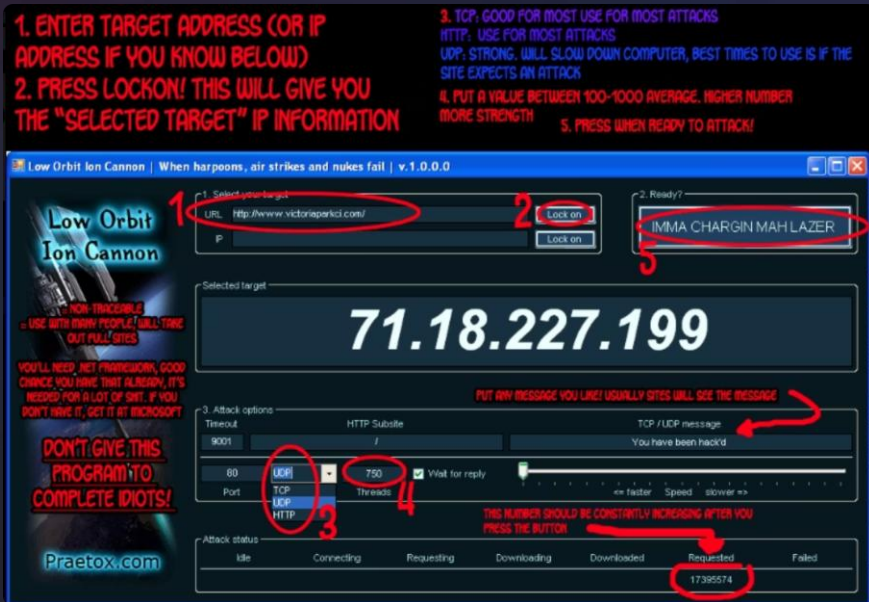
An HTTP Denial of Service tool that uses randomized headers and keep-alive connections to bypass basic rate limiting and exhaust HTTP threads on the target server.

DoS/DDoS Attack Demonstration

The following screenshot illustrates a DoS/DDoS attack scenario in a lab environment. Study the traffic patterns, tool output, and target server responses carefully — these form the basis for understanding detection and mitigation strategies.



Lab Note: Always perform DoS/DDoS testing only in authorized, isolated environments. Launching these attacks against production systems without permission is illegal under computer crime laws in all jurisdictions.



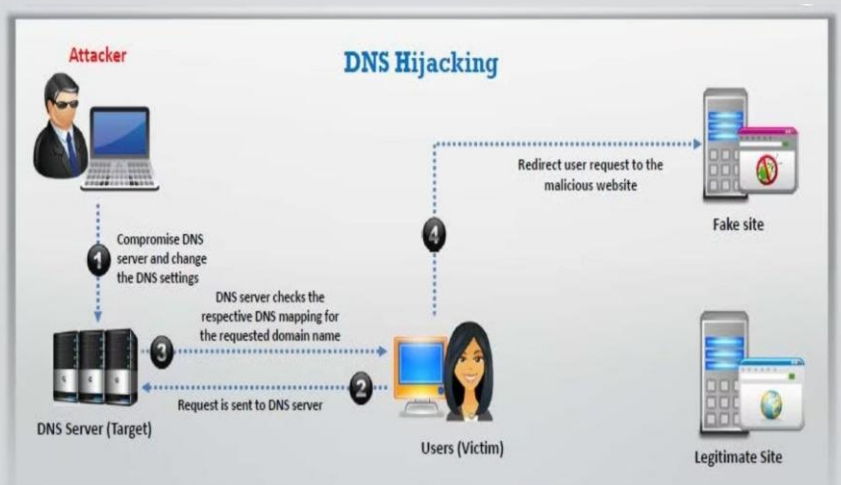
Custom Attack Scenario

Beyond off-the-shelf tools, advanced attackers craft **custom attacks** tailored to specific web server configurations, application logic flaws, or network topologies. These attacks are harder to detect because they do not match known tool signatures and may exploit zero-day vulnerabilities or proprietary weaknesses.

- ❏ Understanding custom attack construction is essential for advanced penetration testers who must simulate realistic threat actors rather than relying solely on automated tooling.

DNS Server Hijacking

DNS hijacking redirects users from legitimate websites to malicious ones by corrupting DNS resolution. An attacker who compromises a DNS server — or poisons its cache — can silently redirect all users querying a domain to an attacker-controlled IP address, enabling credential theft, phishing, and malware distribution at scale.



Cache Poisoning

Injects false DNS records into a resolver's cache, redirecting all queries for a domain.

Rogue DNS Server

Attacker sets up a malicious DNS server and forces clients to use it via DHCP manipulation.

Router Compromise

Modifies the DNS settings on a home or enterprise router to redirect all DNS queries.

DNS Amplification Attack

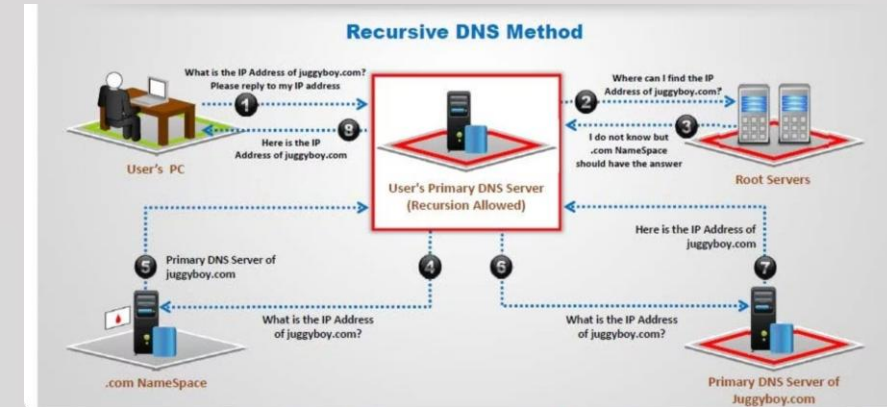
A DNS amplification attack is a type of **reflected DDoS** attack that abuses open DNS resolvers to overwhelm a target with traffic. The attacker spoofs the victim's IP address and sends small DNS queries to multiple open resolvers. Each resolver responds with a much larger reply — directed at the victim — amplifying the attack volume by a factor of 50–100x.

Attack Mechanism

- Attacker spoofs victim's source IP
- Sends small queries to open DNS resolvers
- Resolvers send large responses to the victim

Amplification Factor

- A 40-byte query can return a 4,000-byte response
- Amplification ratio: up to **100x**
- Thousands of resolvers multiply the effect dramatically



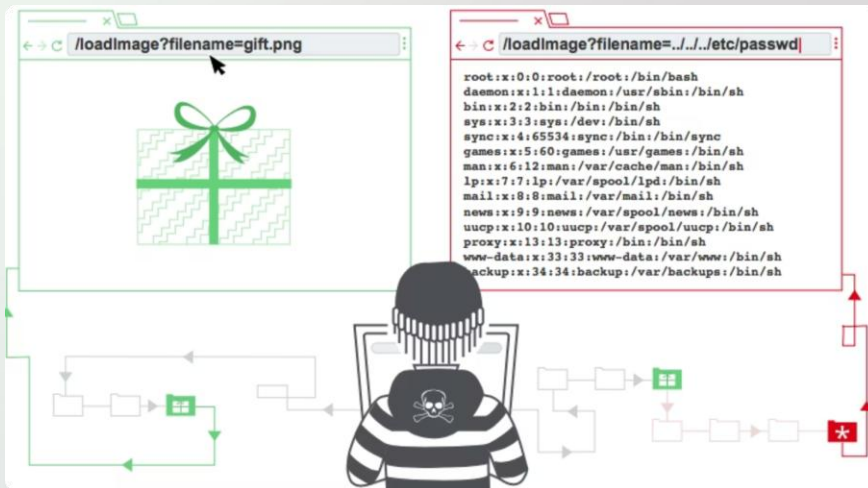


Directory Traversal Attacks — Concept

 **Key Concept:** The web server should restrict all file access to the designated web root. Directory traversal occurs when input sanitization is absent or incomplete, allowing path manipulation to escape this boundary.

Directory Traversal Attacks — In Depth

Directory traversal vulnerabilities are frequently found in web applications that dynamically load files based on user-supplied parameters. Attackers use URL encoding (%2e%2e%2f) or double encoding to bypass basic input filters, reaching sensitive server files outside the intended web directory.



Common Targets

- `/etc/passwd` — Unix user account data
- `/etc/shadow` — Hashed password file
- `web.config` — IIS application configuration
- Application source code and environment files

Bypass Techniques

- URL encoding: `%2e%2e/`
- Double encoding: `%252e%252e/`
- Unicode/UTF-8 variants
- Null byte injection: `%00`

Directory Traversal — Example Attacks

The following screenshot demonstrates real-world traversal attack strings used against a vulnerable web server. Note how path sequences are crafted to step back through the directory structure and reach sensitive operating system files. These examples are critical for understanding both attack construction and detection signatures.



Defense: Implement strict input validation, canonicalize file paths before processing, and use a whitelist of allowed file names. Never pass raw user input directly to file system functions.

```
<?php
$template = 'blue.php';
if ( is_set( $_COOKIE['TEMPLATE'] ) )
    $template = $_COOKIE['TEMPLATE'];
include ( "/home/users/phpguru/templates/" . $template );
?>
```

An attack against this system could be to send the following HTTP request:

```
GET /vulnerable.php HTTP/1.0
Cookie: TEMPLATE=../../../../../../../../etc/passwd
```

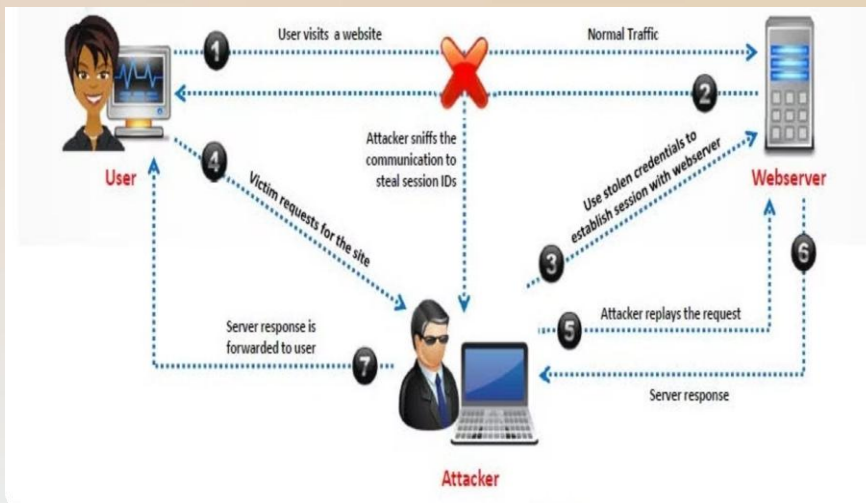
Generating a server response such as:

```
HTTP/1.0 200 OK
Content-Type: text/html
Server: Apache
```

```
root:fi3sED95ibqR6:0:1:System Operator:/:bin/ksh
daemon:*:1:1:/:tmp:
phpguru:f8fk3j10If31.:182:100:Developer:/home/users/phpguru/:bin/csh
```

Man-in-the-Middle / Sniffing Attack

In a Man-in-the-Middle (MitM) attack, the adversary secretly intercepts and potentially alters communications between two parties who believe they are communicating directly. Against web servers, this can result in credential theft, session hijacking, and data manipulation — all without either the client or server being aware.



Interception

Attacker positions themselves between client and server on the network path.

Decryption

SSL stripping or certificate spoofing allows the attacker to read plaintext traffic.

Manipulation

Traffic is modified in transit — injecting malicious scripts or altering responses.

Re-encryption

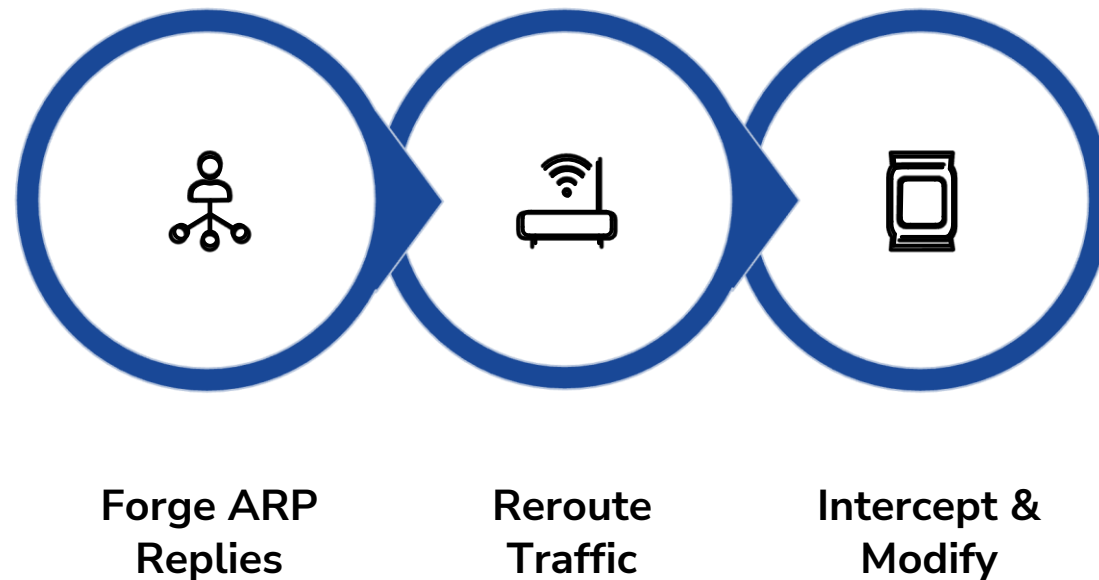
Modified packets are forwarded to the destination, maintaining the illusion of a normal connection.

ARP Spoofing for a MitM Attack

ARP (Address Resolution Protocol) spoofing is one of the most effective techniques for positioning an attacker between two hosts on a local network. By sending forged ARP replies, the attacker convinces the client that the attacker's MAC address belongs to the server — and vice versa. All traffic now flows through the attacker's machine.

❏ **Goal:** Make the client believe we are the server, and make the server believe we are the client — routing all packets through our machine for interception and analysis.

We will use **arp spoof** (part of the dsniff suite) to execute this attack, and then open **BackTrack** / Kali Linux to capture and analyze the intercepted traffic using Wireshark or similar tools.



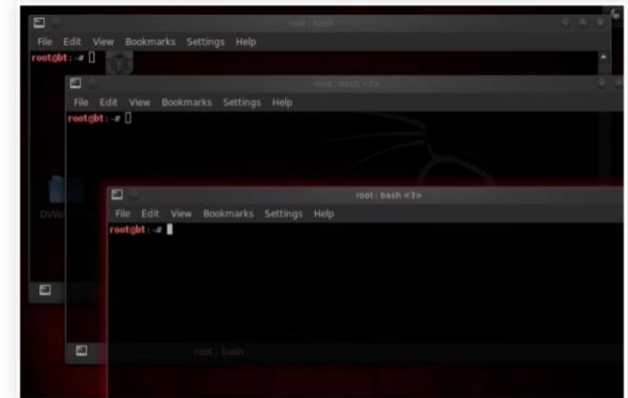
ARP Spoofing — arpspoof in Action

The `arpspoof` tool replaces the legitimate MAC addresses of the client and server in their respective ARP tables with the attacker's MAC address. Once both sides are poisoned, all traffic flows through the attacker's interface, enabling full packet capture.

 **Command syntax:** `arpspoof -i [interface] -t [target IP] [gateway IP]` — run this in two terminals simultaneously to poison both directions of traffic.

Step 1 Open Three Terminals

To conduct this MitM attack, we're going to need three (3) terminals, so go ahead and open those now. Our goal here is to get a client on our network to believe we are the server and the server to believe we are the client.



ARP Spoofing — Enabling IP Forwarding

Step 2 Arpspoof Client to Server

Let's start with the client. We want to replace the MAC address of the server with our MAC address.

- `arpspoof 192.168.1.101 192.168.1.105`

Where:

- `192.168.1.101` is the IP of the client
- `192.168.1.105` is the IP of the server

In this step, we're telling the client that we are the server.

After initiating ARP spoofing, it is essential to enable **IP forwarding** on the attacker's machine so that legitimate traffic continues to flow between the client and server. Without this step, the attack causes a denial of service rather than a transparent interception. The captured traffic can then be analyzed using tools like Wireshark or Ettercap.



Enable IP forwarding on Linux: `echo 1 >`

`/proc/sys/net/ipv4/ip_forward`. This ensures packets are relayed between the poisoned hosts and the attack remains undetected.

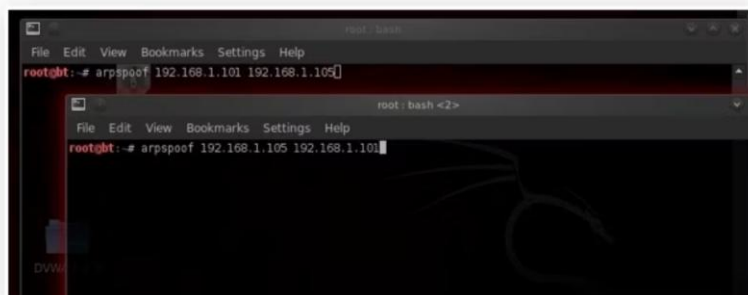
ARP Spoofing — Capturing Traffic

With ARP poisoning active and IP forwarding enabled, the attacker can now capture all traffic passing between the client and server. The screenshots below show the ARP table manipulation and subsequent packet capture, revealing cleartext credentials and session tokens on unencrypted connections.

Step 3 Arpspoof Server to Client

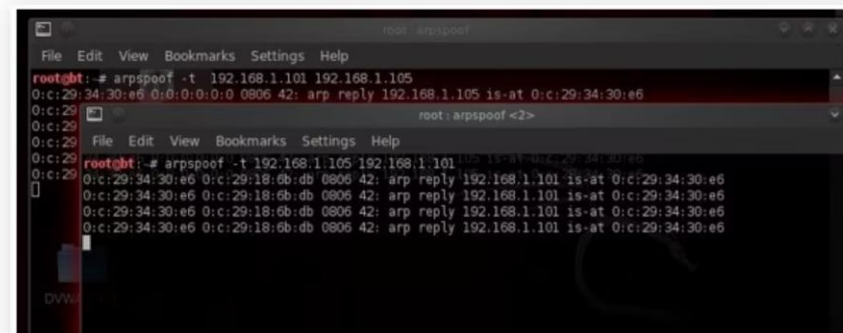
Now we want to replace the MAC address of the client with our address, so we simply reverse the order of the IP addresses in the previous command.

- **arpspoof 192.168.1.105 192.168.1.101**



Here, we are telling the server that we are the client.

Now execute both of these commands. When we do this, the client will think we are the server and the server will think we are the client!



ARP Spoofing — Wireshark Analysis

Wireshark is used to inspect the intercepted traffic in detail. Analysts can filter by protocol, follow TCP streams, and extract credentials from HTTP sessions. The screenshots demonstrate how plaintext login data becomes visible once the MitM position is established — highlighting why HTTPS and certificate pinning are critical defenses.

Step 4 Pass Packets with Ipforward

Now that we are impersonating both the client and server, we need to be able to pass or forward the packets to the other machine. In other words, we want the packets coming from the server to be forwarded to the client and those coming from the client forwarded to the server.

We do this in Linux by using the **ip_forward**. Linux has a built-in functionality to forward packets it receives. By default, it's turned off, but we can turn it on by changing its value to 1(ON).

We simply echo a 1 and direct (>) it to `/proc/sys/net/ipv4/ip_forward`, thereby turning on ipforwarding.

- `echo 1 > /proc/sys/net/ipv4/ip_forward`



Image via wonderhowto.com

Now our system, in the middle, is forwarding the traffic it receives to both ends of this connection, client and server.

ARP Spoofing — Credential Extraction

In the final stage of the ARP spoofing MitM demonstration, captured HTTP POST data reveals form submissions containing usernames and passwords in cleartext. This underscores why deploying HTTPS across all web services — and enforcing HTTP Strict Transport Security (HSTS) — is a non-negotiable security baseline.

Step 5 Sniff the Traffic with Dsniff

Now that we have all the traffic coming from the client to the server and the server to the client going through our computer, we can sniff and see all the traffic!

To do this, we could use a number of different sniffing tools, including Wireshark or tcpdump, but in this case we'll use Dug Song's **dsniff**. Song designed dsniff to sniff out authentication information that appears on the wire in clear text (non-encrypted). So, protocols such as ftp, telnet, HTTP, SNMP, POP, LDAP, etc. can be sniffed off the wire.

To activate dsniff, we simply type:

• dsniff

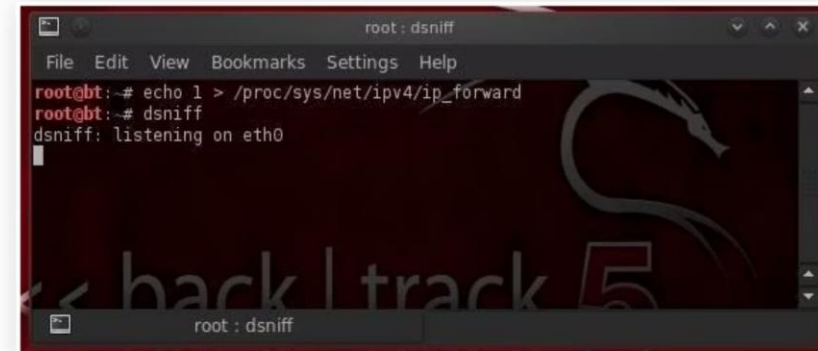
A screenshot of a terminal window titled 'root : dsniff'. The window shows a menu bar with 'File', 'Edit', 'View', 'Bookmarks', 'Settings', and 'Help'. Below the menu, the terminal shows the following commands and output: 'root@bt:~# echo 1 > /proc/sys/net/ipv4/ip_forward', 'root@bt:~# dsniff', and 'dsniff: listening on eth0'. The background of the terminal has a dark red theme with a large, stylized white 'S' logo and the text 'backtrack 5' in a light gray font.

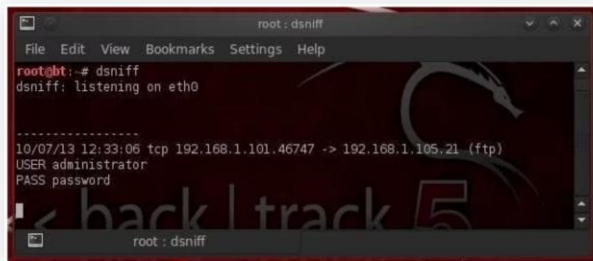
Image via wonderhowto.com

As we can see, dsniff responds that it is listening on eth0.

- ❏ **Defense:** Enforce HTTPS with valid certificates, implement HSTS, use certificate pinning for mobile apps, and deploy Dynamic ARP Inspection (DAI) on managed switches to prevent ARP poisoning.

Step 6 Grab the FTP Credentials

Now, let's wait until the client logs into the ftp server. When he does so, dsniff will grab his credentials and display them to us.

A screenshot of a terminal window titled 'root : dsniff'. The window shows the command 'root@bt:~# dsniff' and the response 'dsniff: listening on eth0'. Below this, a network capture entry is displayed: '10/07/13 12:33:06 tcp 192.168.1.101.46747 -> 192.168.1.105,21 (ftp)'. The captured data shows 'USER administrator' and 'PASS password'. The terminal has a dark background with a red dragon logo and the text 'backtrack 5' in the background.

```
root : dsniff
File Edit View Bookmarks Settings Help
root@bt:~# dsniff
dsniff: listening on eth0

-----
10/07/13 12:33:06 tcp 192.168.1.101.46747 -> 192.168.1.105,21 (ftp)
USER administrator
PASS password
```

Image via wonderhowto.com

As you see in the screenshot above, dsniff has grabbed the ftp credentials of the administrator with the password of "password"! How easy was that!

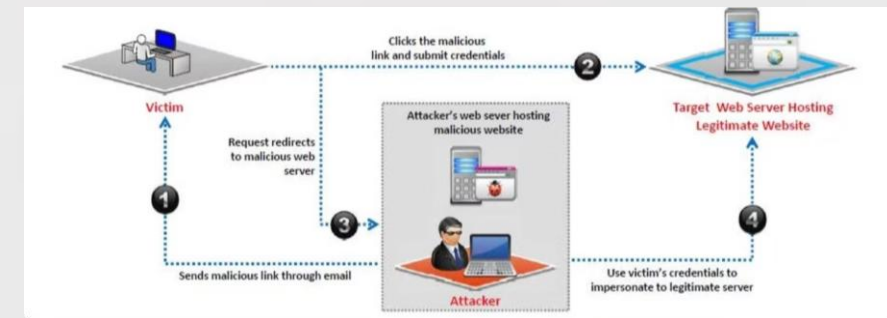
ARP Spoofing — Full Traffic Interception

This screenshot provides a complete view of the intercepted session, showing the attacker's ability to observe bidirectional traffic flows between the victim client and the web server. At this stage, the attacker has full visibility into all unencrypted communications and can optionally inject or modify content in real time.

MitM Outcome: Phishing Attack

Once an attacker has established a MitM position, they can serve a convincing **phishing page** to the victim — intercepting the real web server's response and replacing it with a fake login form. The victim enters credentials believing they are on a legitimate site, while the attacker captures every keystroke in real time.

- Phishing via MitM is highly effective because the URL in the browser address bar may still show the legitimate domain, bypassing users' visual URL checks. SSL stripping removes HTTPS indicators, further deceiving victims.



MitM Outcome: Website Defacement

Website defacement is another consequence of a successful MitM or server compromise. Attackers replace legitimate web content with their own messages — often for political, ideological, or reputational damage purposes. High-profile defacements of government and corporate sites are commonly reported in cybersecurity news.

Attack Goals

- Public embarrassment of the target organization
- Political or ideological messaging ("hacktivism")
- Demonstrating server access to escalate ransom demands

Common Methods

- Exploiting file upload vulnerabilities
- Leveraging compromised CMS credentials
- Remote code execution via unpatched software



Web Server Misconfiguration

Misconfiguration is one of the leading causes of web server compromise. Default installations often leave unnecessary features enabled, sample files accessible, and error messages verbose enough to reveal server internals. Attackers actively probe for these weaknesses during reconnaissance.

Verbose Error Pages

Expose server version, OS, and stack traces — providing a roadmap for targeted exploits.

Directory Listing

Enabled directory browsing reveals file structure and sensitive resources to unauthenticated users.

Default Credentials

Admin interfaces left with factory usernames/passwords are trivially exploited by automated tools.

Unnecessary Modules

Unused server modules expand the attack surface without providing operational benefit.

Web Server Misconfiguration — Example

The following example illustrates a real-world misconfiguration scenario where server banners, default pages, or exposed configuration files provide attackers with critical intelligence. This information is then used to select appropriate exploits or craft targeted payloads against the identified software versions.

This configuration allows anyone to view the **server status** page, which contains detailed information about the current use of the web server, including information about the **current hosts** and requests being processed

httpd.conf file on an **Apache** server

```
<Location /server-status>
SetHandler server-status
</Location>
```

This configuration gives **verbose error messages**



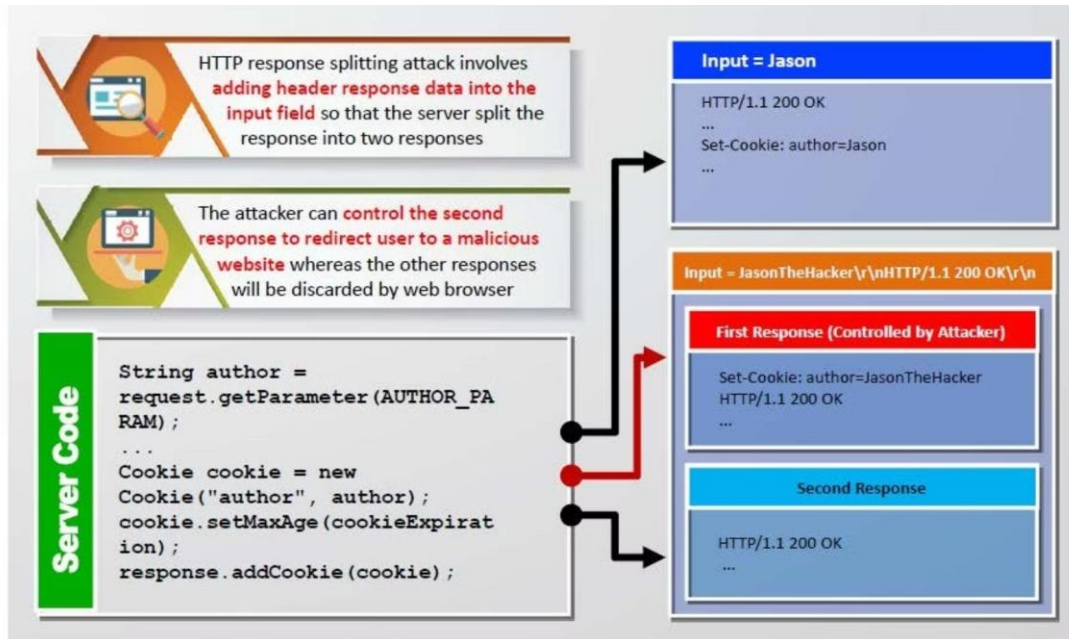
php.ini file

```
display_error = On
log_errors = On
error_log = syslog
ignore_repeated_errors = Off
```

- ❏ **Hardening Tip:** Remove all default pages, suppress server version banners (ServerTokens `Prod` in Apache), disable directory listing, and audit installed modules against a minimum-required baseline.

HTTP Response Splitting Attack

HTTP Response Splitting is a web application attack where an attacker injects **carriage return (CR)** and **line feed (LF)** characters into HTTP headers. Because HTTP uses these characters as header delimiters, the injected content causes the server to generate a split response — creating a second, attacker-controlled HTTP response that can serve malicious content to the victim.



How It Works

- Attacker injects `%0d%0a` (CRLF) sequences into a user-controlled header parameter
- Server includes unvalidated input in the HTTP response header
- The injected CRLF splits the response into two separate HTTP messages

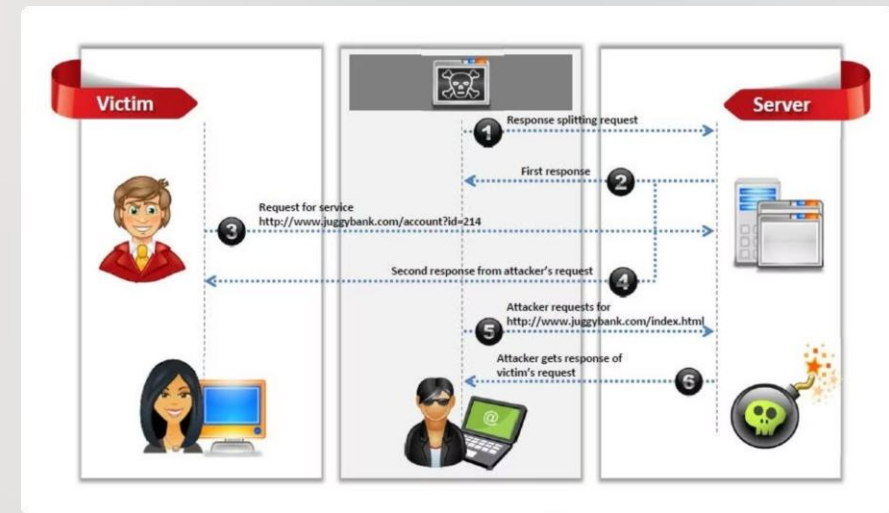
Attack Outcomes

- Cache poisoning — serve malicious content to cached users
- Cross-site scripting (XSS) via injected response body
- Cross-site request forgery (CSRF) facilitation
- Session fixation and credential hijacking

HTTP Response Splitting (cont'd)

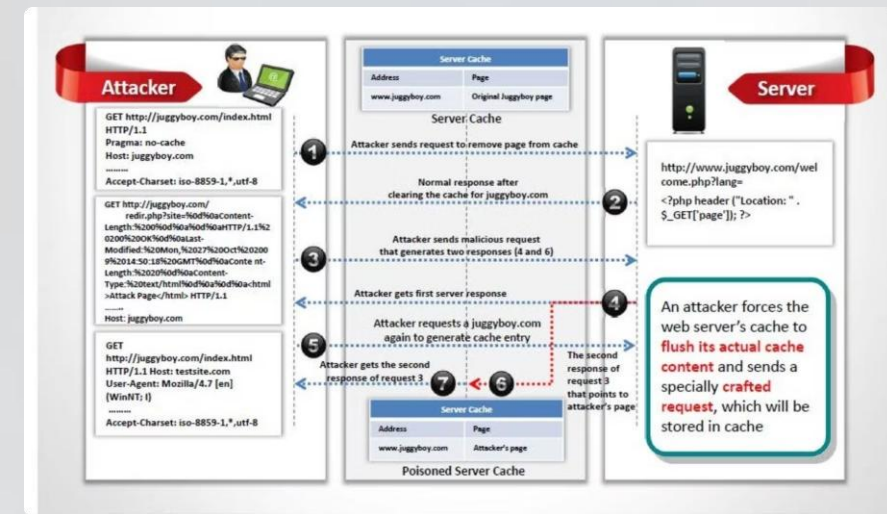
The second part of the HTTP Response Splitting demonstration shows how the attacker's injected content is interpreted by the browser as a legitimate server response. Because the browser trusts the origin of the response, any injected scripts or redirects are executed without warning — making this a potent vector for cache poisoning and cross-site scripting.

- ❏ **Defense:** Validate and sanitize all user input before including it in HTTP response headers. Strip or reject any input containing CR (\r) or LF (\n) characters. Use a Web Application Firewall (WAF) to detect and block CRLF injection attempts.



HTTP Response Splitting (cont'd)

This final view of the HTTP Response Splitting attack illustrates the full impact on downstream caching infrastructure. Intermediate proxies and CDN nodes store the poisoned response and serve it to subsequent legitimate users — multiplying the reach of the attack far beyond the initial victim and making cleanup significantly more complex.



SSH Brute Force Attack

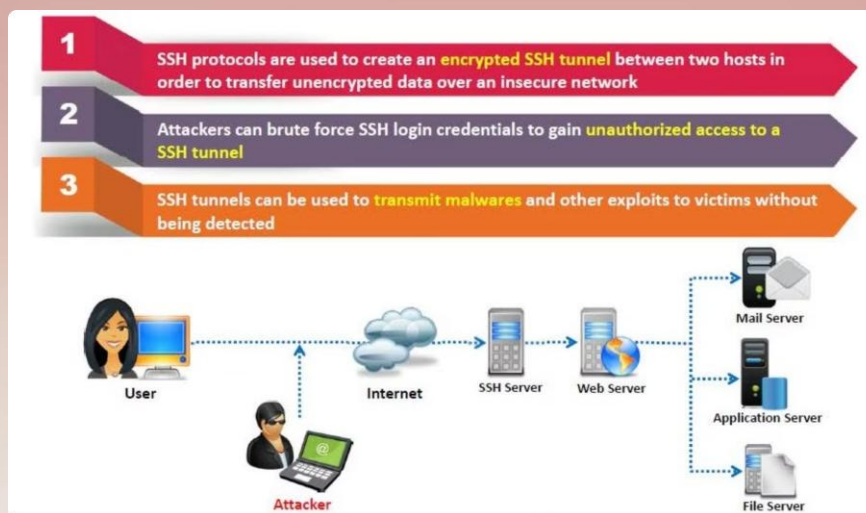
SSH brute force attacks systematically attempt large numbers of username/password combinations against an SSH service to gain unauthorized shell access. Because SSH provides direct command-line control of a server, a successful brute force leads to complete system compromise. Automated tools can attempt thousands of combinations per minute using credential dictionaries.

Common Tools

- **Hydra** — Multi-protocol brute force tool
- **Medusa** — Parallel login brute forcer
- **Ncrack** — High-speed network authentication cracker

Defenses

- Disable password auth; enforce SSH key pairs
- Use **fail2ban** to block IPs after failed attempts
- Change the default SSH port and restrict access by IP



Web Server Password Cracking — Part 1

Beyond SSH, attackers target web-based authentication interfaces, protected admin panels, and application login forms using password cracking techniques. Dictionary attacks, hybrid attacks, and rainbow table lookups are all common approaches. Tools like Burp Suite's Intruder module and Hydra can automate HTTP form-based credential attacks at high speed.

Dictionary Attack

Tries common words and known passwords from wordlists like RockYou or SecLists.

Hybrid Attack

Combines dictionary words with character substitutions (e.g., p@ssw0rd) to expand coverage.

Rainbow Tables

Precomputed hash chains allow near-instant lookup of unsalted password hashes.

Web Server Password Cracking — Part 2

This continuation demonstrates additional cracking methodologies and tool outputs. Particular attention should be paid to how response length and status codes differ between failed and successful authentication attempts — these indicators are what automated tools use to identify valid credentials during a web-based brute force campaign.

- ❏ **Defense:** Implement account lockout policies, CAPTCHA on login forms, multi-factor authentication (MFA), and monitor for repeated failed login events using a SIEM. Strong password hashing (bcrypt, Argon2) mitigates offline cracking of stolen hashes.

Web Application Attacks

Web application attacks target the software layer running on top of the web server — exploiting vulnerabilities in custom code, third-party components, and application logic. Unlike network-layer attacks, these cannot be blocked by a firewall alone, as the malicious payload arrives in legitimate HTTP requests.

SQL Injection

Manipulates database queries via unsanitized input, enabling data extraction, modification, or deletion.

Cross-Site Scripting (XSS)

Injects malicious scripts into web pages viewed by other users, stealing sessions or redirecting browsers.

CSRF

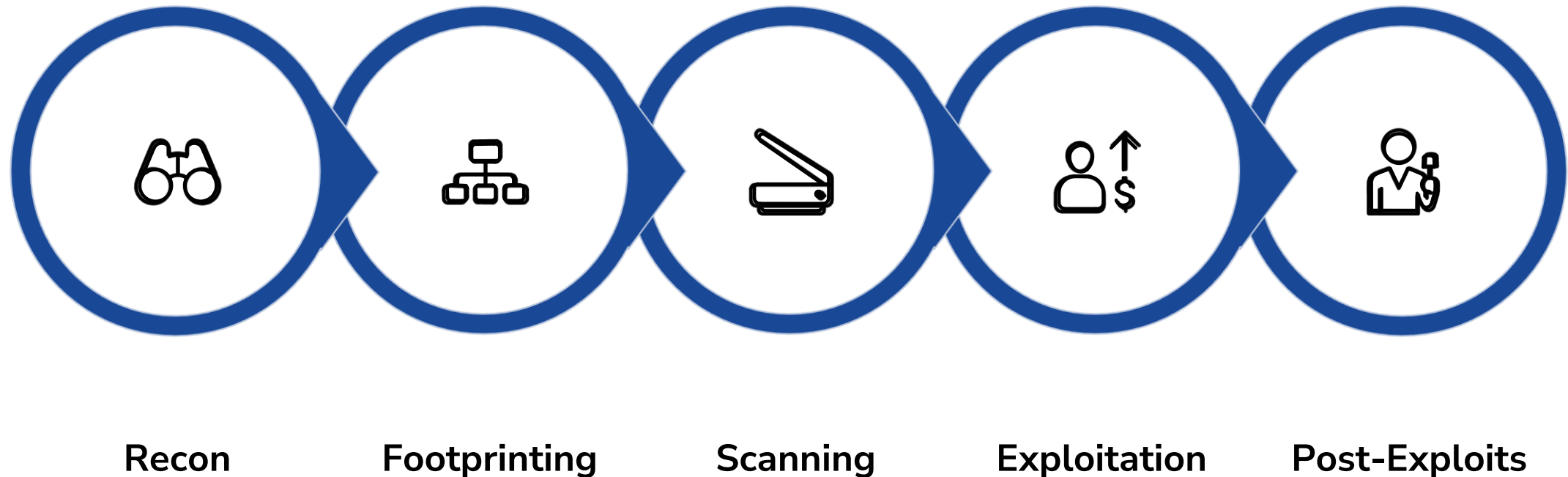
Tricks authenticated users into submitting malicious requests to a trusted web application.

Remote File Inclusion

Forces the server to execute a remotely hosted malicious script, leading to full code execution.

Web Server Attack Methodology

Professional attackers and ethical hackers follow a structured methodology when targeting web servers. This systematic approach ensures comprehensive coverage of the attack surface and mirrors real-world threat actor behavior — making it the foundation for effective penetration testing engagements.



Attack Methodology: Information Gathering

Information gathering is the first and most critical phase of any web server attack. Attackers collect publicly available data about the target — including IP ranges, domain structure, technology stack, employee information, and exposed services — before launching any active probes. This passive reconnaissance minimizes the risk of detection.

Passive Sources

- WHOIS records and DNS zone data
- Google dorking and Shodan searches
- Social media and job postings revealing tech stack
- Wayback Machine for historical site content

Active Techniques

- HTTP banner grabbing (Netcat, curl)
- DNS enumeration (dig, nslookup, fierce)
- SMTP header analysis for internal hostnames
- Web crawling and link extraction

Information Gathering: The robots.txt File

The `robots.txt` file is intended to instruct web crawlers which paths to avoid indexing. However, it inadvertently **advertises sensitive directories** to attackers. Disallow entries often point directly to admin panels, backup directories, configuration files, and internal tools that the site owner wanted to keep out of search engines — but not hidden from attackers who read the file directly.

 **Key Insight:** Never use robots.txt as a security control. Any path listed under `Disallow` is publicly readable. Protect sensitive resources with proper authentication — not by hoping attackers won't look at the robots file.

Attack Methodology: Web Server Footprinting

Web server footprinting involves actively probing the target server to enumerate its software, version numbers, installed modules, and configuration details. This intelligence narrows the attack surface and enables the selection of targeted exploits. HTTP response headers, error pages, and service banners are all rich sources of fingerprinting data.

Banner Grabbing

Extracts server version from HTTP `Server` header using Netcat or curl.

Error Page Analysis

Custom and default error pages reveal server software, OS, and framework versions.

HTTP Method Probing

Testing OPTIONS, PUT, DELETE methods reveals what the server accepts and potentially exposes file write access.

Cookie Inspection

Session cookie names (e.g., `PHPSESSID`, `ASP.NET_SessionId`) identify the backend technology stack.

Web Server Footprinting Tools

A variety of specialized tools automate the footprinting process, enabling rapid and comprehensive enumeration of web server characteristics. Each tool has strengths suited to different phases of reconnaissance — from passive header analysis to active vulnerability mapping.

Netcraft

Online service providing server uptime history, hosting provider, OS, and web server software versions for any domain.

Httprecon

Advanced web server fingerprinting tool that uses multiple probing techniques to identify server software beyond simple banner reading.

ID Serve

Simple utility for identifying web server software and version from HTTP headers and responses.

Nikto

Open-source web server scanner that checks for outdated software, dangerous files, and misconfigurations against a database of known vulnerabilities.

Enumerating Web Server Information Using Nmap

Nmap is the industry-standard network scanner and is equally powerful for web server enumeration. Beyond port scanning, Nmap's scripting engine (NSE) enables automated detection of web server software versions, exposed directories, SSL/TLS configuration weaknesses, and known CVEs — all in a single scan command.

Key Nmap Flags

- `-sV` — Service version detection
- `-A` — Aggressive scan (OS, version, scripts, traceroute)
- `--script http-headers` — Grab HTTP response headers
- `--script http-enum` — Enumerate directories and files
- `--script ssl-cert` — Inspect SSL certificate details

Useful NSE Categories

- `http-*` — Web server enumeration scripts
- `vuln` — Known vulnerability checks
- `auth` — Default credential testing
- `brute` — Brute force authentication scripts

Nmap Scan — Output Analysis

The Nmap scan output below demonstrates a typical web server enumeration result. Key information revealed includes open ports, running service versions, and HTTP headers. This data directly informs the next phases of a penetration test — guiding vulnerability research and exploit selection based on confirmed software versions.

 **Interpretation Tip:** Focus on the `SERVICE` and `VERSION` columns. Cross-reference detected versions against CVE databases (NVD, Exploit-DB) to identify known unpatched vulnerabilities on the target system.

Table 1: Scanning Techniques

Scanning Technique	Syntax	Use
TCP SYN	-sS	Stealth scan
TCP connect()	-sT	Scan without root privileges
FIN	-sF	Stealth scan
Xmas	-sX	Stealth scan
Null	-sN	Stealth scan
Ping	-sP	Identify live hosts
Version Detection	-sV	Identify services
UDP	-sU	Find UDP services
IP Protocol	-sO	Discover supported protocols
ACK	-sA	Identify firewalls
Window	-sW	Advanced ACK scan
RPC	-sR	Information on RPC services
List	-sL	Dummy for test purposes
Idle	-sI	Scan via third party
FTP Bounce	-b	Historic



Thank You

Questions & Discussion

We've covered web server architectures, major attack types, hands-on methodologies, and the tools used by both attackers and defenders. Apply these concepts in your lab exercises and refer to CEH v12 Module 13 for deeper reading before the next session.

Next Steps

Complete Module 13 reading and attempt the lab exercises on web server enumeration and vulnerability scanning.

Assessment Reminder

Review the learning outcomes on slide 3 — all six outcomes are assessable and map to the CEH 312-50 exam objectives.



